Ensambles: bagging, random forests y boosting

Sesión 11 · Módulo 4: Métodos de ensamble

Bootstrap aggregation · Bosques aleatorios · OOB e importancias
AdaBoost · Gradient Boosting · XGBoost

Datos simulados · scikit-learn

Plan de la sesión (≈90 min)

Qué veremos

- **A.** Panorama: tres formas de combinar modelos (bagging, boosting, stacking)
- **B–C. Bagging** y **Random Forests**: bootstrap, reducción de varianza, aleatoriedad de variables, error OOB, importancias y dependencia parcial
- **D. Boosting** y **AdaBoost**: aprendices débiles con pesos adaptativos
- **E. Gradient Boosting**: descenso de gradiente funcional
- **F. XGBoost**: regularización, *column subsampling* y velocidad
- **G.** Comparación: ¿cuándo usar cada variante?

Una sola idea de fondo: un solo árbol es inestable (A16 de la sesión anterior); los ensambles combinan **muchos** modelos simples para obtener uno más estable y preciso — pero bagging y boosting lo logran de formas **opuestas**.

A1 · Tres formas de combinar modelos

Bagging

En paralelo. Entrena B modelos **independientes** sobre remuestreos distintos y **promedia**. Reduce **varianza**.

Ej.: Random Forest

Boosting

En serie. Cada modelo nuevo corrige los **errores** del anterior; se suman con pesos. Reduce **sesgo** (y algo de varianza).

Ej.: AdaBoost, Gradient Boosting, XGBoost

Stacking

Entrena modelos **distintos** (árbol, SVM, kNN, . . .) y un **meta-modelo** aprende a combinar sus predicciones.

No lo veremos hoy a fondo

Hoy nos enfocamos en las dos familias más usadas en la práctica: **bagging/Random Forests** y **boosting** (AdaBoost, Gradient Boosting, XGBoost).

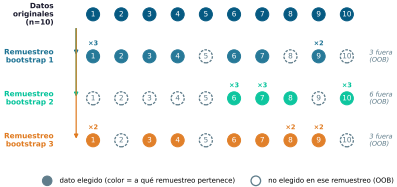
B1 · Bootstrap: remuestrear con reemplazo

La idea

Dado un conjunto de n datos, un **remuestreo bootstrap** toma n datos **al azar, con reemplazo**: algunos aparecen varias veces, otros no aparecen.

Dato clave

En promedio, cada remuestreo deja fuera $\approx \mathbf{36.8}\%$ de los datos originales (ese $\approx 1/e$ es *out-of-bag*, OOB) — lo veremos en C2.



B2 · *Bootstrap aggregating* (bagging): la receta

El algoritmo

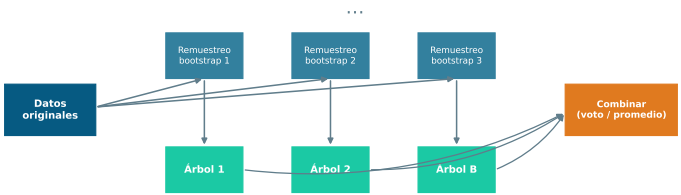
1. Generar B remuestreos bootstrap del entrenamiento.
2. Entrenar un modelo **sin podar demasiado** en cada remuestreo (típicamente árboles, casi sin restringir profundidad).
3. **Combinar** las B predicciones: voto mayoritario (clasificación) o promedio (regresión).

Por qué reduce varianza

Cada modelo individual tiene varianza alta, pero sus errores no están perfectamente correlacionados. Al promediar, los errores **se cancelan parcialmente**: $\mathrm{Var}(\bar{X}) = \frac{\sigma^2}{B}$ si fueran independientes.

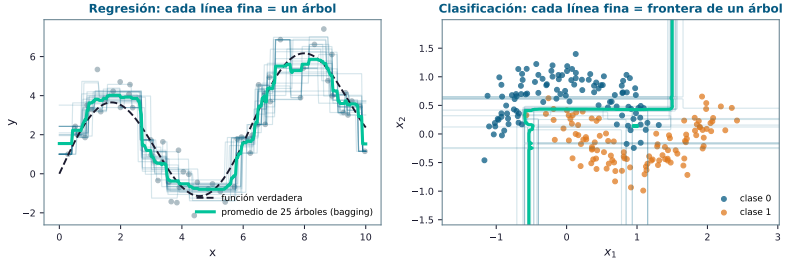
El sesgo de cada modelo individual **no cambia** al promediar — bagging ataca solo la varianza.

B3 · Ilustrando el proceso de bagging



Cada remuestreo bootstrap entrena un modelo distinto (típicamente un árbol poco podado); al final se combinan las B predicciones por voto o promedio.

B3b · Bagging reduce la varianza



Cada línea/frontera delgada es un árbol entrenado en un remuestreo distinto: individualmente son inestables (varianza alta). Al promediar (regresión) o votar (clasificación), las curva/frontera combinada (en verde) es mucho más suave y estable — el sesgo casi no cambia, pero la varianza baja notablemente.

B4 · En scikit-learn: `BaggingClassifier` y `BaggingRegressor`

```python
from sklearn.ensemble import (
    BaggingClassifier, BaggingRegressor)
from sklearn.tree import DecisionTreeClassifier

bagging = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=100,      # cuantos modelos
    max_samples=1.0,       # tamanio de c/remuestreo
    bootstrap=True,        # con reemplazo
    n_jobs=-1, random_state=0)
bagging.fit(Xtr, ytr)
# bagging puede envolver CUALQUIER modelo base,
# no solo arboles (aunque es lo mas comun)
```

Puntos clave

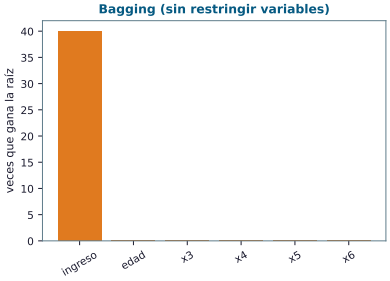
- `estimator` puede ser cualquier modelo base; funciona mejor con modelos de **varianza alta** (árboles sin podar).
- Random Forest (siguiente bloque) es un caso especial de bagging + un ingrediente extra.

C1 · El problema de solo usar bagging

Árboles correlacionados

Si una variable es muy fuerte (como `ingreso` aquí), **casi todos** los árboles bootstrap la usan primero en la raíz: los árboles terminan pareciéndose mucho entre sí.

Promediar árboles **parecidos** reduce poco la varianza: si los errores están correlacionados, $\mathrm{Var}(\bar{X})$ no cae tanto como σ^2/B.



En **40** remuestreos bootstrap sin restringir variables, `ingreso` ganó la raíz en los **40 de 40** árboles.

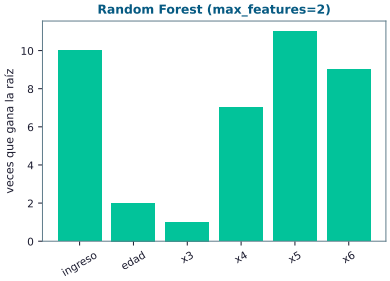
C2 · El ingrediente extra: variables aleatorias en cada corte

La solución de Random Forest

En **cada corte**, en vez de considerar las p variables disponibles, el árbol solo puede elegir entre un subconjunto **aleatorio** de `max_features` variables (típico: $\sqrt{p}$ en clasificación, $p/3$ en regresión).

Dos fuentes de aleatoriedad

(1) **Qué datos** ve cada árbol (bootstrap, como en bagging) y (2) **qué variables** puede usar en cada corte (nuevo).



Con `max_features=2` (de 6 variables), `ingreso` ya solo gana la raíz en **10 de 40** árboles: menos correlación, más diversidad para promediar.

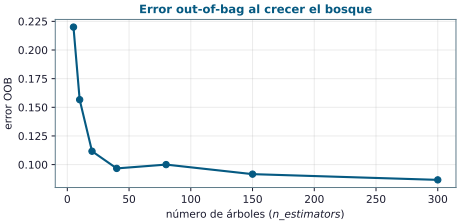
C3 · Error *out-of-bag* (OOB): validación "gratis"

La idea

Cada árbol del bosque deja fuera $\approx 36.8\%$ de los datos (los **OOB** de ese árbol, visto en B1). Para cada dato, se puede predecir usando **solo** los árboles que no lo vieron en entrenamiento.

Por qué importa

El error OOB es casi idéntico al de validación cruzada, pero se obtiene **sin apartar** un conjunto de validación ni reentrenar nada extra.



Con pocos árboles (5) el error OOB es alto y ruidoso (≈22 %); con 300 árboles se estabiliza cerca de su valor final (≈8.7 %).

C4 · En scikit-learn: RandomForestClassifier / Regressor

```python
from sklearn.ensemble import (
    RandomForestClassifier,
    RandomForestRegressor)

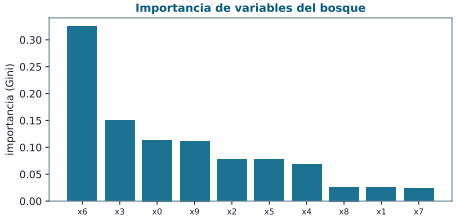
bosque = RandomForestClassifier(
    n_estimators=300,        # cuantos arboles
    max_depth=None,          # casi sin podar
    max_features="sqrt",     # variables por corte
    oob_score=True,          # validacion "gratis"
    random_state=0, n_jobs=-1)
bosque.fit(Xtr, ytr)

bosque.oob_score_              # ~ exactitud de CV
bosque.feature_importances_    # ver C5
```

Puntos clave

- Más árboles (`n_estimators`) casi nunca sobreajusta más; solo cuesta más tiempo.
- Interfaz idéntica a un árbol solo: `fit`, `predict`, `predict_proba`.

C5 · `feature_importances_`: ¿qué variable pesa más?

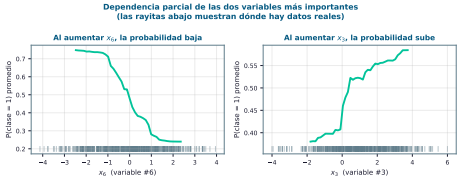


Cómo se calcula

Para cada variable, se suma **cuánto** reduce la impureza (Gini/MSE) en todos los cortes donde se usó, en todos los árboles del bosque, y se normaliza a que sume 1.

Es más **estable** que la importancia de un solo árbol (C1), porque promedia sobre B árboles entrenados con datos y variables distintos.

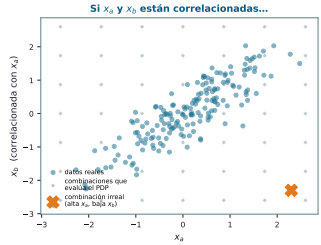
C6 · Dependencia parcial (*partial dependence*)



Las rayitas abajo de cada curva marcan dónde **sí** hay datos reales; donde no hay rayitas, el bosque casi no vio ejemplos ahí y la curva es menos confiable.

Qué responde

"Si **solo** cambiara esta variable (dejando las demás como están, promediando sobre todos los datos), ¿cómo cambiaría la predicción?"



Cuidado: supone variables independientes

Izquierda: los datos reales (azul) solo ocurren en la diagonal, porque x_a y x_b están correlacionadas. El PDP **igual evalúa** toda la rejilla (cruces grises), incluida la "×" roja — una combinación que **nunca** aparece en los datos. Ahí la curva es extrapolación, no algo observado.

C7 · Random Forests: qué se gana y qué se pierde

Se gana

- Mucha menos **varianza** que un árbol solo, con poco ajuste de hiperparámetros.
- `feature_importances_` estable; error OOB sin conjunto de validación aparte.

Se pierde

- Ya **no se puede leer** como reglas si–entonces: se pierde interpretabilidad directa.
- Más lento de entrenar/predecir; sigue sin extrapolar bien fuera del rango de entrenamiento.

En una frase

Un bosque aleatorio cambia interpretabilidad por estabilidad: sigue siendo árboles de decisión por dentro, solo que **muchos**, **diversos** y **promediados**. Bagging y Random Forest reducen **varianza**; no corrigen el **sesgo** de un modelo débil. Para eso está el boosting.

D1 · Boosting: combinar modelos *en serie*

La idea general

En vez de entrenar B modelos **independientes** (bagging), boosting entrena modelos **uno tras otro**: cada modelo nuevo se enfoca en corregir los **errores** que dejó el anterior.

Aprendices débiles

Los modelos base suelen ser muy simples (*weak learners*): árboles de profundidad 1–3 (*stumps*), apenas mejores que adivinar al azar.

La fórmula: $F(x) = \sum_{m=1}^{M} \alpha_m \, h_m(x)$

- M: cuántos aprendices débiles se entrenaron en total (uno por ronda).
- $h_m(x)$: la predicción del aprendiz m para el dato x (p. ej. $+1$ o -1 en clasificación binaria).
- α_m: el **peso de voto** del aprendiz m — cuánto cuenta su opinión en la suma. Un aprendiz que acertó mucho recibe α_m grande; uno que casi no sirvió recibe $\alpha_m \approx 0$ (lo veremos en D2).

No es un promedio simple como en bagging: cada aprendiz **vota distinto** según qué tan bueno fue.

D2 · AdaBoost: el algoritmo completo

Entrada y arranque

n datos (x_i, y_i) con $y_i \in \{-1, +1\}$ (clasificación binaria). Pesos iniciales iguales: $w_i^{(1)} = 1/n$.

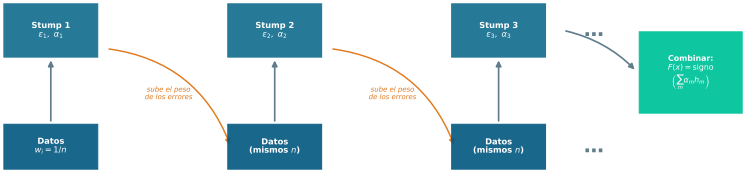
Para cada ronda $m = 1, \ldots, M$

1. **Entrenar** el aprendiz débil $h_m(x) \in \{-1, +1\}$ que minimiza el error **ponderado**: $\varepsilon_m = \sum_{i=1}^{n} w_i^{(m)} \, 1[h_m(x_i) \neq y_i]$
2. **Calcular su voto**: $\alpha_m = \frac{1}{2} \ln \frac{1 - \varepsilon_m}{\varepsilon_m}$ — grande si $\varepsilon_m \to 0$, ≈ 0 si $\varepsilon_m \to 0.5$.
3. **Actualizar pesos**: $w_i^{(m+1)} = w_i^{(m)} \exp\big(- \alpha_m \, y_i \, h_m(x_i) \big)$ — multiplica por e^{α_m} si h_m falló en x_i, por $e^{-\alpha_m}$ si acertó.
4. **Normalizar**: $w_i^{(m+1)} \leftarrow w_i^{(m+1)} / Z_m$, con $Z_m = \sum_i w_i^{(m+1)}$, para que vuelvan a sumar 1.

Al terminar las M rondas, combinar

$$F(x) = \text{signo}\left(\sum_{m=1}^{M} \alpha_m \, h_m(x) \right)$$ — cada aprendiz vota, pero con la fuerza α_m que se ganó.

D2a · Representación ilustrada de AdaBoost



En cada ronda se entrena un *stump* con los datos tal como están pesados; su error ϵ_m define su voto α_m. Los datos que ese *stump* falló quedan con **más peso** para la siguiente ronda (flecha naranja). Al final, los M *stumps* se combinan en una sola votación ponderada.

¿Cómo "entran" los pesos al siguiente *stump*?

Son **los mismos** n **datos** en todas las rondas — a diferencia de bagging, **no se remuestrea**. Lo único que cambia es el vector de pesos $w^{(m)}$, que se pasa directo al entrenamiento (en scikit-learn, `fit(X, y, sample_weight=w)`): el criterio de corte (Gini/entropía ponderado) favorece acertar en los puntos con **peso alto**, aunque sean pocos.

D2a · (continuación) ¿Qué significa pesar en el criterio de corte?

La proporción de clase, ahora ponderada

En un nodo con datos I y pesos w_i, la proporción de la clase k ya no es "cuántos datos de clase k hay" sino **cuánto peso** de clase k hay:

$$p_k = \frac{\sum_{i \in I,\ y_i = k} w_i}{\sum_{i \in I} w_i}$$

El Gini del nodo, con esa p_k ponderada

$$G(I) = 1 - \sum_k p_k^2$$

Al evaluar un corte candidato (I_{izq}, I_{der})

$$G_{\text{corte}} = \frac{\sum_{I_{\text{izq}}} w_i}{\sum_I w_i} G(I_{\text{izq}}) \ + \ \frac{\sum_{I_{\text{der}}} w_i}{\sum_I w_i} G(I_{\text{der}})$$

Un dato con w_i grande pesa como si fueran **varias copias**, sin duplicar ninguna fila: el árbol nunca ve "cuántos" datos hay, solo cuánto peso hay de cada clase a cada lado del corte.

D2b · Un ejemplo numérico: calculando α y actualizando pesos

Paso 1–3: entrenar y pesar el error

$n = 10$ datos, todos con peso inicial $w_i = \frac{1}{10} = 0.1$.
El *stump* clasifica mal a **3** de los 10 datos. Como los pesos empiezan iguales, el error ponderado es simplemente la fracción de errores:

$$\varepsilon_1 = \sum_{i \text{ mal}} w_i = 3(0.1) = 0.3$$

$$\alpha_1 = \tfrac{1}{2} \ln \frac{1 - \varepsilon_1}{\varepsilon_1} = \tfrac{1}{2} \ln \frac{0.7}{0.3} \approx \mathbf{0.424}$$

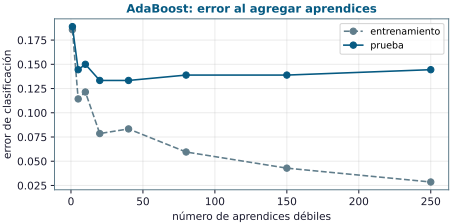
Paso 4: actualizar los pesos

Cada peso se multiplica por $e^{+\alpha_1}$ si el dato se clasificó mal, o $e^{-\alpha_1}$ si se clasificó bien; luego se **normaliza** para que sumen 1.

	7 correctos	**3 incorrectos**
peso antes	0.100	0.100
$\times\, e^{\mp\alpha_1}$	0.0655	0.1528
normalizado	$\mathbf{1/14 \approx 0.071}$	$\mathbf{1/6 \approx 0.167}$

Cada dato mal clasificado queda pesando $\frac{(1/6)}{(1/14)} \approx 2.33$ veces más que uno bien clasificado: el siguiente *stump* se entrena casi obligado a prestarles atención a esos 3 puntos.

D4 · AdaBoost: error al agregar aprendices débiles



Lectura de la curva

Con 1 solo *stump*, el error de prueba es ≈18.9 %; al combinar 250, baja a ≈14.4 %. El error de entrenamiento baja más rápido todavía.

Riesgo

Muy sensible a **datos atípicos y ruido en las etiquetas**: un dato mal etiquetado recibe pesos cada vez más grandes y puede dominar rondas posteriores.

D5 · En scikit-learn: AdaBoostClassifier / Regressor

```python
from sklearn.ensemble import AdaBoostClassifier, AdaBoostRegressor
from sklearn.tree import DecisionTreeClassifier

ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),  # "stump"
    n_estimators=200,
    learning_rate=1.0,      # encoge la contribucion de cada alpha_m
    random_state=0)
ada.fit(Xtr, ytr)
ada.staged_predict(Xte)     # prediccion tras cada ronda (para graficar)
```

`learning_rate` (también llamado *shrinkage*) escala cada α_m: valores más chicos piden más rondas pero suelen generalizar mejor.

E1 · Gradient Boosting: descenso de gradiente *funcional*

La idea

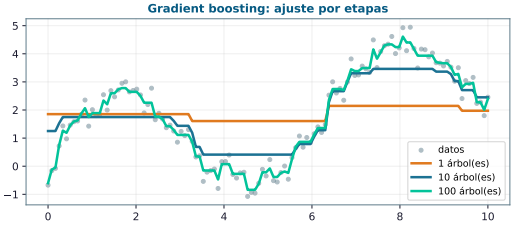
AdaBoost reajusta **pesos de datos**. Gradient Boosting, en cambio, en cada ronda ajusta un nuevo árbol a los **residuos** (el error que falta corregir) del modelo acumulado hasta ahora.

1. Empezar con una predicción simple $F_0(x)$ (p. ej., el promedio de y).
2. Calcular los **residuos** $r_i = y_i - F_{m-1}(x_i)$.
3. Ajustar un árbol pequeño h_m para predecir esos residuos.
4. Actualizar: $F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$.

¿Por qué "gradiente"?

Para la pérdida cuadrática, el residuo **es** (hasta una constante) el gradiente negativo de la pérdida respecto a la predicción. Ajustar al residuo = dar un paso de descenso de gradiente, pero en el **espacio de funciones**.

E2 · Ilustrando el ajuste por etapas



Con 1 árbol la predicción es casi plana; con 10 ya sigue la tendencia general; con 100 (y `learning_rate` pequeño) se ajusta muy de cerca a la curva real — cuidado con el sobreajuste si se exagera.

E3 · En scikit-learn: GradientBoostingClassifier / Regressor

```python
from sklearn.ensemble import (
    GradientBoostingClassifier, GradientBoostingRegressor)

gbm = GradientBoostingClassifier(
    n_estimators=200,
    learning_rate=0.1,    # "eta": que tan grande es cada paso
    max_depth=3,          # arboles pequenios (no stumps, pero chicos)
    subsample=0.8,        # usa 80% de los datos por arbol (menos varianza)
    random_state=0)
gbm.fit(Xtr, ytr)
```

El dial más importante: `learning_rate` vs `n_estimators`

Hay un intercambio directo: `learning_rate` chico + `n_estimators` grande suele generalizar mejor que pasos grandes y pocos árboles — pero tarda más en entrenar.

F1 · XGBoost: gradient boosting, pero más rápido y regularizado

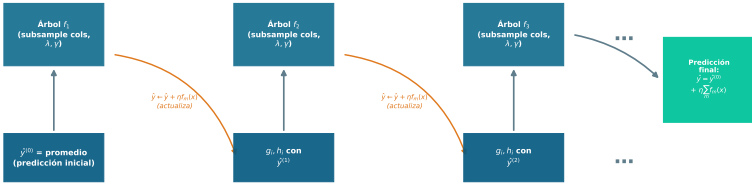
Qué agrega sobre GBM clásico

- **Regularización** L_1/L_2 explícita sobre los pesos de las hojas, directamente en la función objetivo (no solo vía `max_depth`).
- **Column subsampling**: cada árbol (e incluso cada corte) ve solo un subconjunto aleatorio de **columnas**, como en Random Forest.
- Usa **segunda derivada** (información de curvatura) además del gradiente, para pasos más precisos.

Trucos de velocidad

Histogramas: en vez de probar todos los cortes posibles, agrupa los valores en *bins* y prueba solo esos. Soporta **paralelización** dentro de un árbol y manejo nativo de valores **faltantes** (aprende hacia qué lado mandarlos).

F1a · Representación ilustrada de XGBoost



A diferencia de AdaBoost (que repesa **datos**), aquí cada árbol f_m se ajusta a los gradientes g_i, h_i de la pérdida actual — con **column subsampling** y regularización (λ, γ) en cada corte. Tras entrenarlo, la predicción se actualiza con un paso de tamaño η (`learning_rate`), y se repite.

F2 · La función objetivo: aproximación de Taylor de 2do orden

Qué se minimiza al agregar el árbol f_m

$$\mathcal{L}^{(m)} = \sum_{i=1}^{n} \ell\Big(y_i,\ \hat{y}_i^{(m-1)} + f_m(x_i)\Big) \ + \ \Omega(f_m)$$

Se aproxima ℓ con Taylor de 2do orden (más preciso que solo el gradiente)

$$\mathcal{L}^{(m)} \approx \sum_{i=1}^{n} \left[\ell(y_i, \hat{y}_i^{(m-1)}) + g_i\, f_m(x_i) + \tfrac{1}{2} h_i\, f_m(x_i)^2\right] + \Omega(f_m)$$

con $g_i = \dfrac{\partial \ell}{\partial \hat{y}_i^{(m-1)}}$ (gradiente) y $h_i = \dfrac{\partial^2 \ell}{\partial (\hat{y}_i^{(m-1)})^2}$ (hessiano) — el mismo truco de Newton en optimización.

Para pérdida cuadrática: $g_i = \hat{y}_i - y_i$ (el residuo, como en GBM) y $h_i = 1$. XGBoost generaliza esto a **cualquier** pérdida derivable dos veces (p. ej. log-loss en clasificación).

F2a · (continuación) La hoja óptima y la ganancia de un corte

Para una estructura de árbol fija, con hojas $j = 1, \ldots, T$

Agrupando por hoja (I_j = datos en la hoja j), con $G_j = \sum_{i \in I_j} g_i$ y $H_j = \sum_{i \in I_j} h_i$, el peso **óptimo** de esa hoja es:

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

La ganancia de evaluar un corte candidato (I_L, I_R)

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}\right] - \gamma$$

Es el equivalente, en XGBoost, del Gini ponderado de un Random Forest (D2a-cont): mide cuánto **mejora** el ajuste al separar. Si Gain < 0 (el corte no compensa γ), XGBoost **no hace ese corte** — poda integrada en la propia fórmula.

F2b · Un ejemplo numérico: calculando g_i, h_i y la ganancia

Datos y gradientes (pérdida cuadrática)

4 datos, $y = (2, 4, 4, 8)$; predicción actual $\hat{y}^{(0)} = 4.5$ (el promedio) para todos.

$$g_i = \hat{y} - y_i: \quad 2.5,\ 0.5,\ 0.5,\ -3.5$$

$h_i = 1$ para los 4 (pérdida cuadrática)

Corte candidato: izq. $= \{y{=}2, 4\}$, der. $= \{y{=}4, 8\}$.

G, H por lado ($\lambda = 1$)

$$G_L = 2.5{+}0.5 = 3, \quad H_L = 2$$
$$G_R = 0.5{-}3.5 = -3, \quad H_R = 2$$
$$w_L^* = -\tfrac{3}{3} = -\mathbf{1}, \quad w_R^* = -\tfrac{-3}{3} = +\mathbf{1}$$

Ganancia del corte ($\gamma = 0$)

$$\text{Gain} = \tfrac{1}{2}\left[\tfrac{3^2}{3} + \tfrac{(-3)^2}{3} - \tfrac{0^2}{5}\right] - 0 = \tfrac{1}{2}(3 + 3 - 0) = \mathbf{3}$$

El corte separa los y bajos $(2, 4)$ de los altos $(4, 8)$: por eso $w_L^* < 0$ (baja la predicción) y $w_R^* > 0$ (la sube) — justo lo que corrige el error de cada grupo.

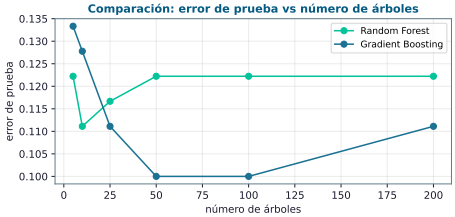
F3 · En XGBoost: XGBClassifier / XGBRegressor

```python
from xgboost import XGBClassifier, XGBRegressor

xgb = XGBClassifier(
    n_estimators=300, max_depth=4, learning_rate=0.08,
    subsample=0.8,          # filas por arbol
    colsample_bytree=0.8,   # columnas por arbol ("column subsampling")
    reg_lambda=1.0,         # regularizacion L2 de las hojas
    reg_alpha=0.0,          # regularizacion L1 de las hojas
    tree_method="hist",     # arboles por histograma: mas rapido
    eval_metric="logloss", random_state=0)
xgb.fit(Xtr, ytr, eval_set=[(Xte, yte)], verbose=False)
```

La interfaz imita a scikit-learn (`fit`/`predict`), por lo que entra sin problema en un `Pipeline` o en `GridSearchCV`.

G1 · Comparación: ¿cuándo usar cada variante?



	Random Forest	**AdaBoost**	**Gradient Boosting / XGBoost**
Ataca	varianza	sesgo (y algo varianza)	sesgo (y algo varianza)
Sensible a ruido/atípicos	poco	**mucho**	moderado (con `subsample`)
Paralelizable entre árboles	sí (completo)	no (secuencial)	no (secuencial)
Ajuste de hiperparámetros	fácil, pocos	moderado	**más delicado**, muchos
Cuándo preferirlo	línea base sólida, rápida	poco usado hoy solo o datos limpios	mejor exactitud, gana competencias (Kaggle)

Actividad de laboratorio (individual)

Bagging, Random Forest y boosting sobre un mismo conjunto de datos

Cuaderno de Jupyter con un conjunto sintético de clasificación. Trabaja **de forma individual**.

1. Entrena un `BaggingClassifier` y un `RandomForestClassifier` con el mismo número de árboles; compara su exactitud y la correlación entre árboles.
2. Grafica el error OOB del bosque contra `n_estimators` y elige un número razonable de árboles.
3. Extrae `feature_importances_` e interpreta cuáles variables pesan más.
4. Entrena `AdaBoostClassifier` y `GradientBoostingClassifier`; grafica el error de prueba contra el número de aprendices para ambos.
5. Ajusta `learning_rate` y `n_estimators` de Gradient Boosting con `GridSearchCV`.
6. Compara los cuatro modelos (bagging, Random Forest, AdaBoost, Gradient Boosting) en una tabla final de exactitud y tiempo de entrenamiento.

Entregable: el cuaderno con las curvas de error OOB y de boosting, la tabla comparativa final y tu interpretación de cuál modelo elegirías y por qué.

Resumen

Las ideas centrales de hoy

- **Bagging**: promediar modelos entrenados en remuestreos bootstrap **independientes** $\Rightarrow$ reduce **varianza**.
- **Random Forest**: bagging + variables aleatorias por corte $\Rightarrow$ árboles menos correlacionados, menor varianza todavía; da OOB e importancias "gratis".
- **Boosting**: combinar aprendices débiles **en serie**, cada uno corrigiendo al anterior $\Rightarrow$ reduce **sesgo**.
- **AdaBoost** reajusta pesos de los **datos**; **Gradient Boosting** ajusta árboles a los **residuos** (descenso de gradiente funcional); **XGBoost** añade regularización, *column subsampling* y velocidad.

En la práctica: Random Forest es casi siempre un buen punto de partida rápido; XGBoost (o LightGBM) suele ganar en exactitud cuando se afinan bien sus hiperparámetros.



¿Preguntas?

¡Pasemos a la actividad de laboratorio: bagging, Random Forest y boosting!